

Adaptive Orchestration of Service Function Chains in SAGIN-MEC via Graph Reinforcement Learning

Peiying Zhang, Shengpeng Chen, Jian Fan, Lizhuang Tan and Chunxiao Jiang

Abstract—Space-air-ground integrated networks (SAGINs) augmented with mobile edge computing (MEC) provide a unified yet heterogeneous substrate for latency-sensitive services. Deploying service function chains (SFCs) over satellites, aerial platforms, and ground nodes, however, is difficult due to hierarchical resource heterogeneity, time-varying network states, and stringent end-to-end (E2E) delay requirements. In this paper, we study online SFC embedding in a three-layer SAGIN-MEC architecture under coupled computing and networking constraints. We model the deployment as a two-stage process: (i) placing each virtual network function (VNF) onto feasible nodes subject to computing-capacity constraints, and (ii) mapping inter-VNF traffic onto feasible paths subject to bandwidth and delay constraints. To achieve adaptive decisions under dynamic states, we cast the problem as graph reinforcement learning by jointly encoding the substrate topology and each SFC into a unified graph state, and propose a structure-aware PPO agent that combines graph convolution with domain features and an action-masking mechanism to eliminate infeasible placement/routing actions. Extensive experiments in dynamic large-scale scenarios show that the proposed method consistently outperforms competitive baselines, improving the acceptance rate by 6.991% under high load, reducing the average E2E delay by 13.556%, and increasing the long-term revenue-to-cost ratio by 27.763% on average.

Index Terms—Deep Reinforcement Learning, Service Function Chain, SAGIN, Mobile Edge Computing, Resource Allocation

This work is partially supported by the National Natural Science Foundation of China under Grant 62341131, 62325108, 62471493 and 62402257, and partially supported by the Natural Science Foundation of Shandong Province under Grant ZR2023LZH017, ZR2024MF066, and 2023QF025, and partially supported by the Open Foundation of Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Qilu University of Technology (Shandong Academy of Sciences) under Grant 2023ZD010. (Corresponding author: Chunxiao Jiang.)

Peiying Zhang is with the Qingdao Institute of Software, College of Computer Science and Technology, China University of Petroleum (East China), Qingdao 266580, China and with the Shandong Key Laboratory of Intelligent Oil & Gas Industrial Software, Qingdao 266580, China and with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Qilu University of Technology (Shandong Academy of Sciences), Jinan, 250014, China (e-mail: zhangpeiying@upc.edu.cn).

Shengpeng Chen is with the Qingdao Institute of Software, College of Computer Science and Technology, China University of Petroleum (East China), Qingdao 266580, China and with the Shandong Key Laboratory of Intelligent Oil & Gas Industrial Software, Qingdao 266580, China (e-mail: z24070039@s.upc.edu.cn).

Fan Jian is with the School of Automation Science and Electrical Engineering, Beihang University, Beijing 100191, China. (e-mail: 15810101661@163.com)

Lizhuang Tan is with the Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan, 250014, China and with the Shandong Provincial Key Laboratory of Computing Power Internet and Service Computing, Shandong Fundamental Research Center for Computer Science, Jinan, 250014, China (e-mail: tanlzh@sdas.org)

Chunxiao Jiang is with the Beijing National Research Center for Information Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: jchx@tsinghua.edu.cn).

I. INTRODUCTION

WITH the continuous advancement of satellite communications, low-altitude unmanned platforms, and terrestrial network technologies, the Space-Air-Ground Integrated Network (SAGIN) is emerging as a core infrastructure for next-generation ubiquitous connectivity. It has been widely applied in scenarios such as disaster emergency communications [1], [2], remote medical services [3], and vehicular networks [4]. For example, when large-scale natural disasters paralyze terrestrial communication base stations, satellites and aerial platforms can quickly restore regional communication links. Similarly, in remote mountainous or maritime regions where ground network coverage is absent, satellite and aerial nodes can collaborate to provide continuous connectivity. These typical use cases share common demands: end-to-end (E2E) network services with low latency, high reliability, and strong resilience.

However, traditional SAGIN architectures primarily focus on data forwarding and connectivity, lacking sufficient computational resources and intelligent scheduling mechanisms to ensure quality of service (QoS) [5]. To address the computational and real-time processing requirements of emerging applications, Mobile Edge Computing (MEC) has been introduced as a key enabling technology within the SAGIN framework. It enables satellites, unmanned aerial vehicles (UAVs), and terrestrial edge nodes to host computing resources and provide near-source processing services [6]. For instance, low Earth orbit (LEO) satellites equipped with edge computing modules can support video recognition and route planning for ocean-going vessels, while edge-enabled UAVs can assist disaster response teams with AI-based image processing and environmental modeling. Therefore, SAGIN enhanced by MEC (SAGIN-MEC) leads to a heterogeneous, converged computing network.

Within this context, the Service Function Chain (SFC) has become a critical mechanism for supporting diverse demands in data processing, security enforcement, and content distribution. SFC enables the ordered deployment of virtual network functions (VNFs) (e.g., NAT, IDS, and video transcoding along network paths), thereby achieving logical function orchestration at the application layer. However, such mechanisms are heavily dependent on the unified control and resource abstraction capabilities provided by software defined networking (SDN) and network function virtualization (NFV) architectures [7]. As illustrated in Figure 1, the control plane virtualizes the multilayer physical network into a unified resource graph and executes SFC placement, mapping, and scheduling. Nonetheless, this idealized control mechanism faces substantial challenges in the SAGIN-MEC environment.

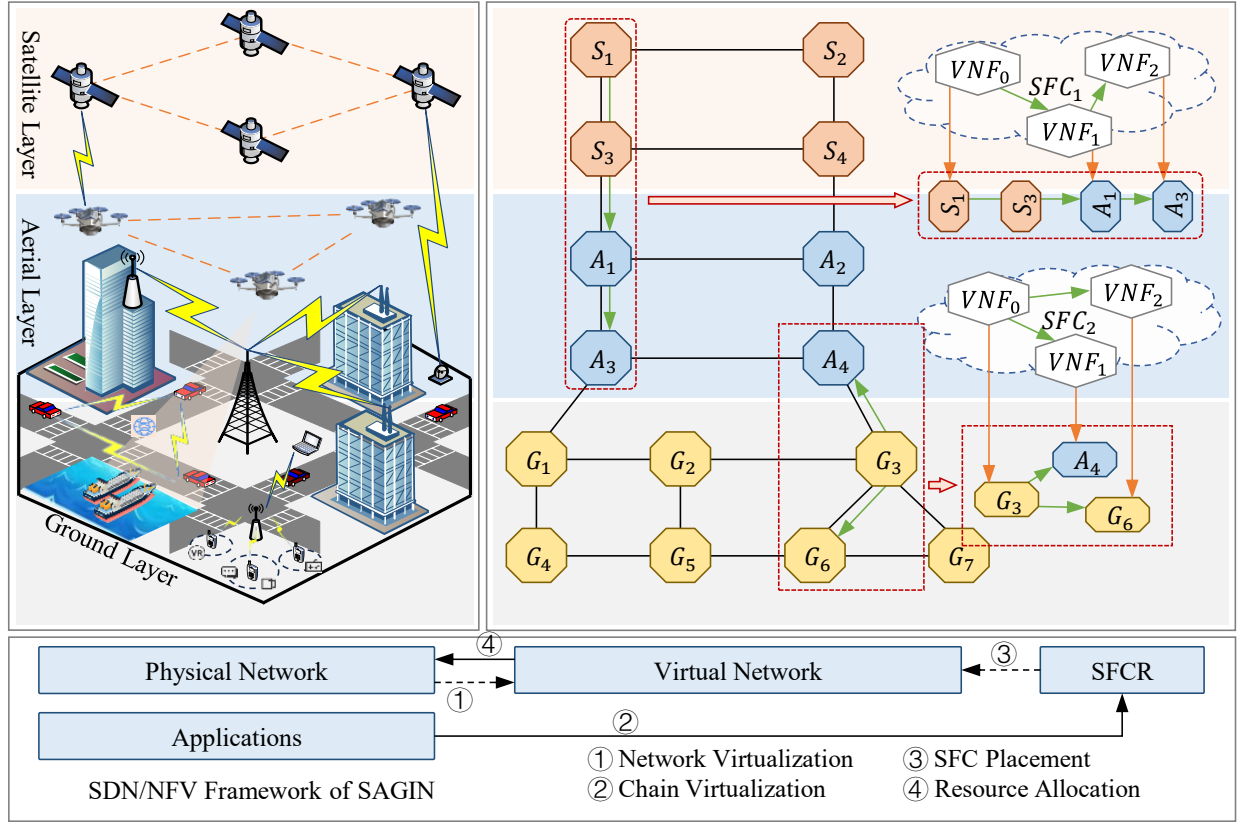


Fig. 1: Schematic diagram of the SAGIN-MEC architecture. Symbols S , A , and G in the figure represent the Satellite, Aerial, and Ground segments, respectively, and S_i, A_i, G_i denote the i -th node within each segment. For example, the SFC_2 to be deployed carries VNF instances (VNF_0, VNF_1, VNF_2) with a topology structure ($VNF_0 \rightarrow VNF_1, VNF_0 \rightarrow VNF_2$). Starting with VNF_0 , the physical node G_3 is selected to host VNF_0 based on policy. In the second decision round, VNF_1 is deployed to A_4 , and in the third round, VNF_2 is deployed to G_6 . Subsequently, link mapping is performed. If the bandwidth resources satisfy the requirements of the virtual links, ($VNF_0 \rightarrow VNF_1$) is mapped to ($G_3 \rightarrow A_4$), and ($VNF_0 \rightarrow VNF_2$) is mapped to ($G_3 \rightarrow G_6$). Among multiple paths for link mapping, the one with the shortest hop count is selected. As a result, the final activated physical network topology becomes ($G_3 \rightarrow A_4, G_3 \rightarrow G_6$).

First, the heterogeneity across network layers is significant. Satellite links offer broad coverage but suffer from high latency; aerial links are flexible in deployment but less stable; terrestrial links provide high capacity but are locally constrained [5], [8]. Service routing must dynamically balance bandwidth, latency, and reliability across these heterogeneous links. Second, the network topology is highly dynamic, with frequent changes in node positions and severe link state fluctuations, which pose difficulties for the SDN controller in terms of delayed information and global synchronization. Furthermore, SFC deployment must strictly adhere to VNF sequencing, computing resource constraints, and link bandwidth limitations, thereby making the embedding process extremely complex. As shown by SFC_1 and SFC_2 in Figure 1, certain VNFs must be deployed across domains and layers on different types of nodes, while path mapping must concurrently satisfy bandwidth checks and sequencing constraints. Thus, SFC embedding is no longer a simple resource matching task but requires joint modeling and optimization of node embedding, link mapping, and service path selection.

To address these issues, various approaches have been

proposed. Heuristic algorithms and Integer Linear Programming (ILP)-based methods [9]–[11] have demonstrated effectiveness in small-scale, static networks; however, they suffer from scalability and real-time limitations in the highly dynamic and large-scale SAGIN-MEC environment. Recently, Graph Convolutional Network (GCN) [12] have been widely adopted in SFC-related tasks due to their advantages in modeling complex topologies and extracting structural features. Reinforcement learning techniques have also shown promising adaptability in learning embedding policies. Nevertheless, these methods often lack integration, failing to simultaneously address topological modeling and policy optimization. Moreover, existing works have yet to effectively model the multi-domain, heterogeneous, and dynamic nature of the SAGIN-MEC environment.

To overcome these challenges, this paper proposes a joint SFC embedding framework based on GCN and Proximal Policy Optimization (PPO). We first utilize GCN to perform perception-driven modeling of the multi-layer heterogeneous topology in SAGIN-MEC, extracting the state features of nodes and links. Then, a PPO-based deep reinforcement learning (DRL) algorithm is employed to learn adaptive embedding

strategies within the continuously evolving state space. This enables the joint optimization of resource allocation (RA) and path selection, thereby maximizing SFC embedding success rates while minimizing E2E communication latency. The main contributions of this work are summarized as follows:

- Addressing the challenges of heterogeneity and dynamism, we formulate the orchestration of SFCs in a three-layer SAGIN-MEC architecture as a graph reinforcement learning (GRL) problem. Specifically, we represent each SFC request as a directed acyclic graph (DAG) and embed it together with the heterogeneous SAGIN topology into a unified graph state, on top of which the agent jointly decides VNF placement and routing paths under coupled computing and networking constraints.
- We develop a structure-aware PPO algorithm whose policy network combines graph convolutional layers with carefully designed node and edge features, including the relative position of VNFs in the chain and their distance to already placed functions. To efficiently handle hard capacity, ordering, and connectivity constraints, we incorporate a domain-specific action masking scheme that explicitly filters out infeasible placement and routing actions before sampling, which reduces the effective action space and stabilizes training compared with vanilla PPO.
- We implement a configurable simulation environment that emulates a hierarchical SAGIN with dynamic workloads and heterogeneous MEC resources, and instantiate a broad set of heuristic and DRL-based baselines within this environment. Extensive experiments show that the proposed method consistently improves SFC acceptance ratio and E2E delay while achieving a higher long-term revenue-to-cost ratio.

The rest of this paper is organized as follows: Section II presents the related works on SFC deployment in SAGIN-MEC environments. Section III describes the system model, including network design, SFCR definition, and problem formulation. Section IV details the proposed GCN-PPO-based SFC embedding framework. Section V presents the experimental results and analysis. Finally, Section VI concludes the paper.

II. RELATED WORKS

The deployment of SFC remains a major challenge in SAGIN-MEC environments due to their inherent dynamism and heterogeneity. Existing studies on this topic can be broadly categorized into two main research directions: (1) heuristic and optimization-based, and (2) DRL-based. Additionally, recent trends focusing on SAGIN-specific strategies and graph learning-based models have garnered increasing attention.

A. Heuristic and Optimization-Based Approaches

Initial research on the SFC deployment problem primarily relied on heuristic algorithms and ILP models. These methods demonstrate satisfactory resource utilization and interpretability, particularly in small-scale, static networks. For example, Wang et al. [10] proposed a greedy strategy based on node attribute heterogeneity, employing a dynamic weighting function to optimize the trade-off between communication and computation

resources in SAGIN. Zhang et al. [11] introduced a service-priority-based RA mechanism that enhances deployment success rates for mission-critical applications. Sasabe et al. [9] modeled the SFC mapping problem as an ILP to achieve theoretically optimal solutions. Addressing the dynamic re-scheduling and remapping of SFCs in SAGIN, Li et al. [13] employed a tabu search algorithm to obtain near-optimal ILP solutions. He et al. [14] proposed an online joint offloading and power control framework based on a two timescale time expanded graph to maximize long term offloaded data under energy constraints, while they further formulated a joint scheduling and power-allocation problem and develops approximation and hybrid algorithms to balance total energy consumption and task makespan [15]. Despite the high deployment accuracy and controllability of these methods, their significant computational complexity and limited adaptability hinder their applicability to real-world SAGIN-MEC scenarios characterized by rapid node mobility, frequent topological changes, and large-scale concurrent requests.

B. DRL-Based SFC Deployment Optimization

With the advancement of deep neural network, researchers have begun leveraging DRL for dynamic decision-making in resource optimization. Compared to traditional optimization approaches, DRL offers enhanced adaptability to dynamic environments and supports online policy updates. For instance, Liu et al. [16] employed the DDPG to construct an SFC deployment framework for dynamic mapping in complex cloud networks. Zhu et al. [17] proposed a multi-agent DDGP-based solution to enable cooperative deployment of multiple service chains in NFV environments, improving both scalability and flexibility. Subramanya et al. [18] combined RL with heuristic methods to optimize user access and SFC mapping strategies in 5G networks, resulting in improved overall system performance. However, these approaches often neglect the structural information of network topologies and lack explicit modeling of heterogeneous network architectures. In multi-domain, hierarchical deployment scenarios, strategy optimization without structure-awareness is insufficient to capture the complex and dynamic behavioral patterns inherent to SAGIN systems.

C. Deployment Optimization for SAGIN-MEC Environments

SAGIN integrates heterogeneous nodes including satellites, high-altitude platforms, UAVs, and terrestrial infrastructures, and is typically characterized by a multi-domain, hierarchical, and time-varying architecture. As illustrated in Figure 1, the space layer is commonly modeled as a time-evolving satellite network, where satellites may be interconnected via a limited number of inter-satellite links subject to payload, pointing, and link-budget constraints. In the air layer, UAVs can form an ad hoc/self-organized network and obtain backhaul connectivity through satellites, high-altitude platforms, or terrestrial gateways, depending on deployment and mission requirements. The ground layer is usually supported by relatively stable fiber/core networks and cellular access networks, whose topology and link states evolve at a slower timescale compared with the

non-terrestrial segments. Cross-layer links are intermittently available and depend on satellite/UAV positions, visibility (e.g., line-of-sight), channel conditions, beam alignment, and resource constraints (e.g., per-node interface limits and link capacities), leading to a heterogeneous, hierarchical, and time-varying three-dimensional network [19]. These attributes pose significant challenges for traditional deployment strategies. To capture the spatiotemporal dynamics of such systems, Cao et al. [20] developed a reconfigurable time-expanded graph model representing satellite coverage periodicity, UAV mobility, and the stability of terrestrial nodes. Recent studies further emphasize online SFC scheduling under time-varying SAGIN topologies and multi-task contention. For instance, Jia et al. [21] model urgent and unexpected tasks that introduce conflicts during SFC deployment and propose a reconfigurable time-extension graph to capture mobility-induced dynamics, then formulate an SFC scheduling problem that maximizes the number of successfully deployed SFCs within limited resources and time horizons, and solve it with deep reinforcement learning. Zhang et al. [22] introduced a priority-based dynamic resource reservation protocol to alleviate resource contention caused by low-priority tasks. In study [23], the author leveraged SDN to support VNF migration, facilitating load balancing and QoS assurance across domains. While these strategies provide tailored solutions for SAGIN scenarios, most rely on static modeling or rule-based scheduling mechanisms, lacking real-time adaptability and coordinated awareness. Consequently, they fall short in addressing frequent environmental changes and complex inter-node interactions. An additional challenge in SAGIN lies in resource heterogeneity. Wang et al. [24] proposed a heterogeneous CPU+DPU architecture to enhance service acceptance rates and reduce latency during SFC deployment. Xia et al. [25] explored the use of LEO satellites as edge servers and developed a latency-aware deployment algorithm that significantly reduced the average request delay. Yang et al. [26] focused on cross-geographical data centers, aiming to achieve low-latency and cost-effective SFC deployment. Beyond resource-aware embedding at the data plane, space-terrestrial integrated SFC orchestration also faces control-plane challenges due to fast LEO topology dynamics. Zheng et al. [27] propose the geosynchronous SFC scheme by clustering adjacent LEO satellites into logical VNF containers anchored at fixed positions, so that ground controllers can maintain space-terrestrial SFCs without being affected by continuously shifting satellite VNFs, avoiding large-scale routing policy recalculation.

D. Emerging Trends of Graph Neural Network for SFC Deployment

Graph Neural Network (GNN), particularly Graph Convolutional Network, have demonstrated strong capabilities in modeling complex network structures and have been widely applied in tasks such as link prediction, node classification, and policy learning. Their inherent strength in capturing structured features and inter-node relationships makes them suitable for deployment optimization in dynamic and heterogeneous environments. Recent studies [28]–[30] have attempted to

integrate GCN into network RA to enhance policy generalization through topology awareness. For example, Wang et al. [28] combined GCN with multi-agent reinforcement learning to address dynamic SFC deployment. However, their method was constrained by unidirectional action selection. To overcome this limitation, Wang et al. [29] introduced a bi-directional MDP model, improving exploration flexibility in the action space. Fan et al. [30] proposed a DRL framework augmented with GCN-based feature extraction and employed experience replay to accelerate convergence. The QoS in SFC deployment is heavily influenced by the number of VNFs and the complexity of network topology, making it challenging to meet resource constraints and delay requirements in dense environments. Zhang et al. [31] addressed the startup latency introduced by NFV infrastructures by incorporating GCN-based topology-aware caching mechanisms. Their design enables cache decisions without relying on prior deployment knowledge, enhancing adaptability. Despite the promising capabilities of GCN, most existing studies treat GCN as auxiliary tools for feature extraction rather than integrating them deeply with policy learning. For instance, Wang et al. [32] used GCN to extract physical network features and developed a sequence-to-sequence model to generate deployment strategies. However, the strategy generation itself was conducted by a conventional RL network, limiting the integration of topological modeling with policy optimization. Moreover, current research often focuses on single-domain or static-topology environments, failing to provide unified models that address the multi-domain, heterogeneous, and dynamic characteristics of SAGIN-MEC systems. Thus, combining the structural awareness of GCN with the adaptive policy learning capabilities of DRL to build a unified deployment framework remains a key challenge and a promising research direction.

III. SYSTEM MODEL

In this section, we present the system model that underpins our proposed approach, integrating the definition of the SAGIN, the characteristics of Service Function Chain Requests (SFCR), and the formal problem formulation. The key notations used throughout this section are summarized in Table I.

A. Network Design

We model SAGIN as a three-layer network where each layer constitutes an independent domain $\{\mathcal{S}, \mathcal{A}, \mathcal{G}\}$ corresponding to space, aerial, and ground segments respectively. The physical infrastructure within SAGIN comprising physical nodes and interconnections, is formally represented as an undirected graph $G\{N, L\}$, where $N = N^{\mathcal{S}} \cup N^{\mathcal{A}} \cup N^{\mathcal{G}}$ denotes the union of heterogeneous computing node sets and $L = \{L^{\mathcal{S}} \cup L^{\mathcal{A}} \cup L^{\mathcal{G}} \cup L^{\mathcal{S} \leftrightarrow \mathcal{A}} \cup L^{\mathcal{S} \leftrightarrow \mathcal{G}} \cup L^{\mathcal{A} \leftrightarrow \mathcal{G}}\}$ represents substrate links. Each physical node $n_i \in N$ possesses three key attributes: computational capability $RES_{cpu}^{max}(n_i)$, storage capability $RES_{mem}^{max}(n_i)$, and processing latency $DLY_{proc}(n_i)$. Actually, these computational nodes are interconnected via wireless links or wired cables. We abstract physical link as a connection defined by a source and a destination and also characterized by two attributes: bandwidth $RES_{bw}^{max}(l_i)$ and propagation latency $DLY_{prop}(l_i)$.

TABLE I: Summary of Notations

Network	
$G\{N, L\}$	Physical network graph
N^S, N^A, N^G	Set of physical node.
L^S, L^A, L^G	Set of Physical link within same domain.
$L^{S \leftrightarrow A}, L^{S \leftrightarrow G}, L^{A \leftrightarrow G}$	Set of Physical link within different domain.
$G'\{N', L'\}$	Virtual directed acyclic graph
Resource	
$\Phi_{k,t}$	VNF instances of $SFC_{k,t}$.
$RES_{cpu}^{max}(n_i)$	CPU capacity of physical node n_i .
$RES_{mem}^{max}(n_i)$	Memory capacity of physical node n_i .
$RES_{cpu}^{rem}(n_i)$	Remaining CPU of node n_i .
$RES_{mem}^{rem}(n_i)$	Remaining memory of node n_i .
$RES_{bw}^{max}(l_i)$	Bandwidth capacity of link l_i .
$RES_{bw}^{rem}(l_i)$	Remaining bandwidth of link l_i .
$RES_{cpu}^{req}(n'_i)$	CPU requested by virtual node n'_i (link l'_i).
$RES_{mem}^{req}(n'_i)$	Memory requested by virtual node n'_i .
$RES_{bw}^{req}(l'_i)$	Bandwidth required by virtual link l'_i .
Binary Indicator	
$X_{n'_j \rightarrow n_i}^N$	Node mapping feasibility indicator.
$X_{l'_j \rightarrow l_i \in \hat{L}}^L$	Link mapping feasibility indicator.
$SN_{SFC_{k,t}}$	Node mapping success for $SFC_{k,t}$.
$SL_{SFC_{k,t}}$	Link mapping success for $SFC_{k,t}$.
$\Omega_{SFC_{k,t}}$	Deployment result of $SFC_{k,t}$.
Performance Metric	
T_{occu}	Duration of resource occupation.
$DLY_{proc}(n_i)$	Processing delay at node n_i .
$DLY_{prop}(l_i)$	Propagation delay of link l_i .
$DLY_{SFC_{k,t}}$	Delay of $SFC_{k,t}$.
$\overline{DLY}$	Average delay over accepted SFCs.
$ACCR$	Acceptance rate of SFCs.
$REV_{SFC_{k,t}}$	Revenue of $SFC_{k,t}$.
$COST_{SFC_{k,t}}$	Cost of $SFC_{k,t}$.
$\overline{LRCR}$	Average long-term revenue-cost ratio.

B. Definition of SFCR

Let $SFC_{k,t}$ denote the k -th service chain arriving at time t within request queue $SFCR$, formally defined as $SFC_{k,t} = \{G', \Phi\}_{k,t}$. Each SFC is represented by a directed acyclic graph $G'_{k,t}\{N', L'\} = \{[n'_1, n'_2, \dots, n'_{|N'|}], [l'_1, l'_2, \dots, l'_{|L'|}]\}$, and a set of VNF instances $\Phi_{k,t} = \{\phi_1, \phi_2, \dots, \phi_{|N'|}\}$. To simplify system modeling, we define the i -th VNF ϕ_i to be placed on the i -th virtual node n'_i according to the DAG structure. In addition, $RES_{cpu}^{req}(n'_i)$ or ϕ_i and $RES_{mem}^{req}(n'_i)$ denote the resource attributes required by VNFs and $RES_{bw}^{req}(l'_i)$ is the required bandwidth to ensure data transmission between adjacent VNFs.

C. Problem Formulation

The objective of SFC embedding problem is to establish a feasible mapping from virtual elements to physical infrastructure:

$$\begin{aligned} \Omega : \lim_{\forall t \in [1, T], T \rightarrow \infty} \Omega_{SFC_{k,t}}, \\ \text{s.t. } \Omega_{SFC_{k,t}} : G'_{k,t}\{N', L'\} \xrightarrow{\Phi_{k,t}} G\{N, L\}. \end{aligned} \quad (1)$$

Proccession $\Omega_{SFC_{k,t}}$ usually be divided into two steps [33], containing node selection and link mapping. For each virtual node $n' \in N'$, a substrate node $n \in N$ is selected with sufficient residual resources. For a group of VNFs within the

same SFC, each physical node can only be selected once and host at most one VNF instance. Physical links, however, can be mapped multiple times and a physical link mapped by multiple virtual links consumes bandwidth equal to the maximum virtual bandwidth, as resources are allocated sequentially over time. Intuitively, it is impossible to allocate VNF to physical nodes with insufficient resource resources, and at any time t within system lifespan $[1, T]$, the resources of physical nodes and links are non negative, resulting in the following constraints:

$$RES_*^{rem} \leftarrow \begin{cases} RES_{cpu}^{rem}(n_i, t) = RES_{cpu}^{max}(n_i) - \sum_{n'_j \in H(n_i, t)} RES_{cpu}^{req}(n'_j), \\ RES_{mem}^{rem}(n_i, t) = RES_{mem}^{max}(n_i) - \sum_{n'_j \in H(n_i, t)} RES_{mem}^{req}(n'_j), \\ RES_{bw}^{rem}(l_i, t) = RES_{bw}^{max}(l_i) - \sum_{l'_j \in H(l_i, t)} RES_{bw}^{req}(l'_j), \end{cases}$$

s.t. for $\forall t \in [1, T]$, $RES_*^{rem}(\cdot, t) \geq 0, * \in \{cpu, mem, bw\}$, (2)

where $H(\cdot, t)$ denotes all virtual nodes(links) are hosting on physical node n_i (link l_i) at time t . Based on resource constraints, the binary indicator X is used to represent the verification results of node mapping and link mapping:

$$X_{n'_j \rightarrow n_i}^N = \begin{cases} 1, & \text{if } \begin{pmatrix} RES_{cpu}^{rem}(n_i, t) \geq RES_{cpu}^{req}(n'_j) \\ RES_{mem}^{rem}(n_i, t) \geq RES_{mem}^{req}(n'_j) \end{pmatrix}, \\ 0, & \text{otherwise.} \end{cases} \quad (3)$$

$$X_{l'_j \rightarrow l_i \in \hat{L}}^L = \begin{cases} 1, & \text{if } \left(\exists \hat{L} \subseteq L, \text{ s.t. } \min_{l_i \in \hat{L}} [RES_{bw}^{rem}(l_i, t)] \geq RES_{bw}^{req}(l'_j) \right), \\ 0, & \text{otherwise.} \end{cases} \quad (4)$$

Firstly, the resource constraints of the nodes need to be verified. If the resource constraints of the first stage are met, the link bandwidth constraints of the second stage need to be further verified. The satisfaction of two-stage constraints can be calculated as follows:

$$SN_{SFC_{k,t}} = \begin{cases} 1, & \text{if } \left(\sum_{n'_j \in N'} \sum_{n_i \in N} X_{n'_j \rightarrow n_i}^N = |N'| \right), \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

$$SL_{SFC_{k,t}} = \begin{cases} 1, & \text{if } \left(\exists \hat{L} \in L, \sum_{l'_j \in \hat{L}} X_{l'_j \rightarrow l_i}^L = |\hat{L}| \right), \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

Then, when all virtual nodes and links meet the deployment conditions, it is considered that $SFC_{k,t}$ is acceptable. The deployment result of $SFC_{k,t}$ is calculated as follows:

$$\Omega_{SFC_{k,t}} = \begin{cases} 1, & \text{if } (SN_{SFC_{k,t}} \wedge SL_{SFC_{k,t}} = 1), \\ 0, & \text{otherwise.} \end{cases} \quad (7)$$

Once $SFC_{k,t}$ is accepted, the computing and bandwidth resources it occupies will be exclusively allocated. Other requests cannot occupy these resources again until the $SFC_{k,t}$ is actively released at some time $t' > t$ in the future.

D. Metrics for Performance

1) *Acceptance Rate (ACCR)*: The acceptance rate metric calculates the number of SFCs that are accepted and successfully embedded into the substrate network, representing the correctness of the evaluated algorithm. Additionally, *ACCR* is defined as the ratio of accepted SFCs to the total number of arrivals, which can be calculated as follows:

$$ACCR = \lim_{T \rightarrow \infty} \frac{\sum_{SFC_{k,t} \in SFCR} \Omega_{SFC_{k,t}}}{|SFCR|}. \quad (8)$$

2) *Average Delay ($\overline{DLY}$)*: The delay of an SFC is calculated as the sum of the time consumption of each VNF and link, which can be formulated as:

$$DLY_{SFC_{k,t}} = \sum_{i=1}^{|N|} \sum_{j=1}^{|N'|} X_{n'_j \rightarrow n_i}^N \times DLY_{proc}(n_i) + \sum_{i=1}^{|L|} \sum_{j=1}^{|L'|} X_{l'_j \rightarrow l_i}^L \times DLY_{prop}(l_i). \quad (9)$$

Then, the average delay of all successfully embedded SFCs can be defined as follows:

$$\overline{DLY} = \lim_{T \rightarrow \infty} \frac{\sum_{SFC_{k,t} \in SFCR} \Omega_{SFC_{k,t}} \times DLY_{SFC_{k,t}}}{\sum_{SFC_{k,t} \in SFCR} \Omega_{SFC_{k,t}}}. \quad (10)$$

3) *Average Long-term Revenue-Cost Ratio ($\overline{LRCR}$)*: Deploying SFC on edge servers incurs additional costs, significantly increasing resource consumption and running expenses. To balance resource utilization and cost, we adopt the *revenue-cost* model to evaluate network performance. Specifically, the revenue generated by SFC embedding is the total amount of computing resources it brings, while the cost is modeled as the peak reserved bandwidth footprint of this SFC on physical links during the same period. Accordingly, the revenue and cost of $SFC_{k,t}$ are given by:

$$REV_{SFC_{k,t}} = T_{occu} \left[\sum_{n'_i \in N'} (RES_{cpu}^{req}(n'_i) + RES_{mem}^{req}(n'_i)) \right]. \quad (11)$$

$$COST_{SFC_{k,t}} = T_{occu} \left[\sum_{l \in L} \left(\max_{l' \in \{L' \cap H(l,t)\}} RES_{bw}^{req}(l') \right) \right]. \quad (12)$$

The $\overline{LRCR}$ measures the time-averaged revenue-cost ratio over the horizon, computed as

$$\overline{LRCR} = \frac{1}{T} \sum_{SFC_{k,t} \in SFCR} \frac{REV_{SFC_{k,t}}}{COST_{SFC_{k,t}}} = \frac{1}{T} \times \sum_{SFC_{k,t} \in SFCR} \frac{\sum_{n'_i \in N'} (RES_{cpu}^{req}(n'_i) + RES_{mem}^{req}(n'_i))}{\sum_{l \in L} (\max_{l' \in \{L' \cap H(l,t)\}} RES_{bw}^{req}(l'))}. \quad (13)$$

IV. ALGORITHM DESIGN

In this section, we introduce the details of the proposed SFC embedding algorithm. We model the SFC embedding problem with MDP, which transforms the service chain deployment problem in an SDN/NFV framework into a mathematical model that can be solved through RL approach.

A. Problem Modeling with MDP

1) *State S*: To facilitate state definition, the state set is partitioned according to request arrival time as $\mathbf{S} = \{\mathbf{S}_0, \mathbf{S}_1, \dots, \mathbf{S}_T\}$. Specifically, each timeslot state $\mathbf{S}_t$ can be further classified by different requests as $\mathbf{S}_t = \{S_{1,t}, S_{2,t}, \dots, S_{k,t}\}$. Each state $S_{k,t}$ comprises $|\Phi_{k,t}|$ sub-states, defined as $S_{k,t} = \{s(\phi_1, t), s(\phi_2, t), \dots, s(\phi_{|N'|}, t)\}$. The network base state is represented by a resource matrix $M = [RES_{cpu}^{rem}, RES_{mem}^{rem}, RES_{bw}^{rem}] \in \mathbb{R}^{|N| \times 3}$. For each VNF $\phi_j \in \Phi_{k,t}$, the state is constructed based on the resource and topology information of the substrate network. In this work, we focus on the following five key features:

- $RES_{cpu}^{rem}(n_i)$: The remaining CPU resources of node n_i are used to evaluate its capacity to accommodate the resource demands of VNF ϕ_j . This metric ensures that the node has sufficient computational resources for the task.
- $RES_{mem}^{rem}(n_i)$: Memory is another critical computational resource. Some VNFs require substantial memory storage, while others prioritize parallel computing. The remaining memory resources of node n_i , denoted as $RES_{mem}^{rem}(n_i)$, are used to assess its suitability for hosting VNF ϕ_j .
- $RES_{bw}^{rem}(L(n_i))$: $L(n_i)$ represents the set of edges directly connected to n_i . and the average remaining bandwidth of node n_i is calculated as the mean of the bandwidths of all links directly connected to it. For VNF ϕ_j , this metric characterizes the node's connectivity, with higher adjacent bandwidth resources increasing the likelihood of successful embedding.
- $DEG(n_i)$: The degree of node n_i is defined as the number of links directly connected to it. Nodes with lower degrees are more susceptible to localized overload issues when assigned high-load tasks due to their limited connection paths. Such nodes may experience traffic congestion if tasked with data traffic exceeding their capacity, so we incorporate node degree as a key dimension in the multidimensional state feature set to quantitatively analyze network characteristics.
- $DIST(n_i, n_{pred})$: To minimize delays, adjacent VNFs in an SFC should be deployed in close proximity. Therefore, we compute the distance between the candidate physical node n and the predecessor's deployed physical node. Specifically, let n_{pred} denote the physical node hosting the predecessor of the current VNF in the SFC graph; the distance feature is defined as $DIST(n, n_{pred})$. If the current VNF has no predecessor (i.e., it is a source VNF), we set $DIST(n, n_{pred}) = 0$.

Then, the state for VNF ϕ_i at time t can be constructed as follow:

$$s(\phi_i, t) = \left[RES_{cpu}^{rem}(n_i), RES_{mem}^{rem}(n_i), RES_{bw}^{rem}(L(n_i)), DEG(n_i), DIST(n_i, n_{pred}) \right]. \quad (14)$$

Considering that node resource features and topological features lie in very different numerical ranges, we rescale the raw resource features by $RES = \frac{RES}{100}$ before feeding them into the network, so that prevents large-scale features from dominating the gradient updates in the hidden layers and leads

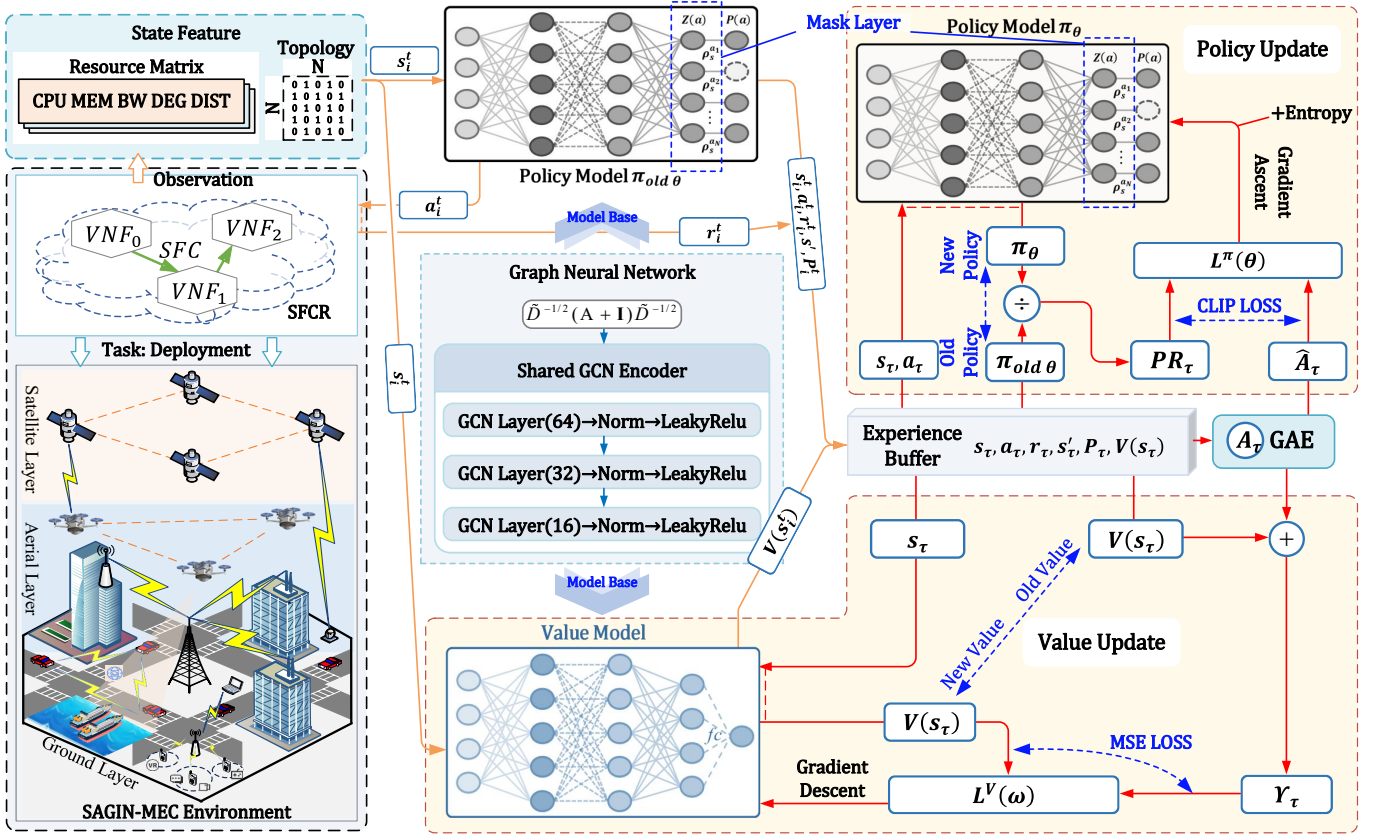


Fig. 2: Design of GCN-based policy and value models and the overall training pipeline.

to a more balanced optimization. Since this is a simple linear rescaling applied consistently in both training and evaluation, it does not change the underlying semantics of the features.

2) **Action A**: The action space $\mathbf{A}$ is defined as the set of all possible actions that can be taken by the agent. In our approach, the size of action space is equal to the number of available physical nodes by $\mathbf{A}_t = \{A_{1,t}, A_{2,t}, \dots, A_{k,t}\}$. For whole $SFC_{k,t}$, a sequence of actions $A_{k,t} \subseteq \mathbf{A}_t$ is generated as $A_{k,t} = \{a(\phi_1, n_{q_1}), a(\phi_2, n_{q_2}), \dots, a(\phi_{|N'|}, n_{q_{|N'|}})\}$, where q represents a combination of node indexes with length $|N'|$, denoting the series of continual actions taken by agent. Action $a(\phi_i, n_{q_i})$ represents VNF ϕ_i carried by virtual node n'_i is embedding to physical node n_{q_i} . The nodes selected for any action $a(\phi_i, n_{q_i})$ in $A_{k,t}$ are different, meaning that any VNF instance in the same SFC cannot be embedded on the same physical node. For the action sequence $A_{k,t}$ corresponding to SFC, the dimension is equal to the number of VNFs contained in SFC.

3) **Reward R**: The reward is the instantaneous feedback signal provided by the environment to the agent, serving to evaluate the quality of actions taken by the agent in specific state $s(\phi_i, t)$. The reward set can be defined as $\mathbf{R} = \{\mathbf{R}_0, \mathbf{R}_1, \dots, \mathbf{R}_T\}$. For $SFC_{k,t}$, a group of consecutive rewards $R_{k,t} \subseteq \mathbf{R}_t$, $R_{k,t} = \{r_1^t, r_2^t, \dots, r_{|N'|}^t\}$ can be observed and we formulate the reward based on an *Revenue-Cost* model proposed in Eq.(11) and Eq.(12). Reward r_i^t is defined as the ratio of the benefits and costs of an action. Here, we calculate the reward set $R_{k,t}$ for $SFC_{k,t}$ based

on the adopted actions, that is, the benefits of using action embeddings are compared to the overall SFC embedding cost:

$$\begin{aligned}
 r_i^t &= f_i^t \times \frac{REV(\phi_i)}{COST_{SFC_{k,t}}}, \\
 &= f_i^t \times \frac{RES_{cpu}^{req}(n'_i) + RES_{mem}^{req}(n'_i)}{\sum_{l \in L} (\max_{l' \in \{L' \cap H(l,t)\}} RES_{bw}^{req}(l'))}, \quad (15) \\
 \text{s.t. } f_i^t &= \begin{cases} +1 & \text{if } \Omega_{SFC_{k,t}} = 1, \\ -1 & \text{otherwise,} \end{cases}
 \end{aligned}$$

B. Model Design

This section will detail the design of the model present in Figure 2, including: (1) the policy model, and (2) the value model. Specifically, the policy model processes the state matrix and guides the agent in executing actions, while the value model addresses instability issues during the parameter updates of the policy network.

1) **Policy Model**: The policy model is designed to facilitate action selection while ensuring compliance with resource and positional constraints through a masking mechanism. The input to the network is the state matrix, as defined in Eq.(14). The policy network adopts a five-layer architecture, as illustrated in Figure 2. Specifically, the input layer receives the state with dimensionality F . The convolutional layer then applies an initial linear transformation followed by three hidden GCN blocks with 64, 32 and 16 units, respectively, each activated by the Leaky ReLU function to enhance non-linear feature

extraction. Next, the mask layer incorporates resource and feasibility constraints by suppressing invalid actions in the score space. The probability layer normalizes the masked scores with a softmax operation to obtain a valid action distribution. Finally, the output layer generates the selected action, and the corresponding raw action score vector is denoted as $Z(\cdot)$.

To ensure the validity of actions, a masking layer is incorporated to exclude actions that violate resource or positional constraints, as specified in Eq.(3) and Eq.(5). The valid action scores are subsequently normalized using a softmax function, producing the final action probability distribution. This architecture ensures that the policy model not only adheres to the defined constraints but also effectively captures the underlying network dynamics for optimal decision-making.

$$\pi_\theta(a | s) = \sigma(Z(GCN(\hat{s})) + \rho_s^a), \quad (16)$$

$$\text{s.t. } \hat{s} = \tilde{D}^{-1/2}(\mathbb{A} + \mathbf{I})\tilde{D}^{-1/2}s,$$

where $\mathbb{A}$ represents the adjacency matrix of the physical network, $\mathbf{I} \in \mathbb{R}^{N \times N}$ denotes the identity matrix, $\sigma(\cdot)$ is the softmax function and ρ_s^a is defined as the mask vector, which assigns a value of $-\infty$ to invalid actions and 0 to valid ones. $\tilde{D}$ represents the degree matrix, defined as $\tilde{D} = \text{diag}(d_1, d_2, \dots, d_{|N|})$, where $d_i = \sum_{j=1}^{|N|} [\mathbb{A} + \mathbf{I}]_{i,j}$. The policy model outputs a probability distribution over all feasible actions, denoted as $P = \pi_\theta(a|s)$, which quantifies the probability of selecting action a at the given state s .

2) *Value Model*: The value network shares the same input as the policy network and processes it through a three layer GCN blocks followed by Leaky ReLU activation and a final fully connected layer producing a scalar output representing the estimated value $V(s(\phi_i, t))$ of the current state $s(\phi_i, t)$.

C. Proposed Algorithm for SFC Embedding based on PPO

1) *Formulation*: Let π_θ denotes the current policy, which takes the state matrix $s(\phi_i, t)$ as input and outputs an action probability distribution. The agent executes virtual node embedding operations based on the output of the policy π_θ . Once all virtual nodes have been embedded into the physical network, link mapping operations are performed. An SFC is considered successfully embedded only when all virtual nodes and virtual links are successfully mapped to the physical network. The environment computes an instantaneous reward corresponding to each action taken. For a group of VNFs $\{\phi_1, \phi_2, \dots, \phi_{|N'|}\}$ within $SFC_{k,t}$, a example of trajectory can be expressed as $\langle s(\phi_i, t), a_i(\phi_i, n_i), r_i^t, s(\phi_{i+1}, t) \rangle$. For example, as illustrated in Figure 1, after deploying the chain SFC_2 , the environment observes three sets of trajectory information: $\langle s(\phi_1, t), a_1(\phi_1, G_3), r_1^t \rangle$, $\langle s(\phi_2, t), a_2(\phi_2, A_4), r_2^t \rangle$, and $\langle s(\phi_3, t), a_3(\phi_3, G_6), r_3^t \rangle$. Using the observed trajectories, the parameters of the policy network and value network are iteratively updated to improve the embedding strategy. For the k -th batch of observed trajectories $\{\tau_1, \tau_2, \dots, \tau_k\}$, the collected data can be expressed as:

$$\tau_k = \{s_\tau, a_\tau, r_\tau, s'_\tau\}, \quad (17)$$

$$= \{\langle s_i^t, a_i^t, r_i^t, s_{i+1}^t | s_0^{t+1} \rangle \mid t \in [kT, (k+1)T - 1]\},$$

where s' represents the next state to be transferred after executing action a_i^t at state s_i^t . Then, the one-step Temporal

Algorithm 1 Training Process of PPO-Based SFC Embedding

```

1: Input:  $\gamma, \lambda, \alpha_\pi, \alpha_V, \epsilon, \text{batch\_size}, N_{\text{epoch}}, \dots$ 
2: Initialize actor  $\pi_\theta$ , critic  $V_\omega$ , buffer  $B \leftarrow \emptyset$ 
3: for episode = 1 to  $N_{\text{episodes}}$  do
4:   Reset environment
5:   for each  $SFC_{k,t}$  in SFCR do
6:      $SN_{SFC_{k,t}}, SL_{SFC_{k,t}} \leftarrow \text{true}; \mathbf{S}_t, \mathbf{A}_t, \mathbf{P}_t \leftarrow \emptyset$ 
7:     for each VNF  $\phi_i$  in  $SFC_{k,t}$  do
8:        $s_i \leftarrow s(\phi_i, M_t)$ 
9:        $P_i \leftarrow \pi_\theta(\cdot | s_i)$ 
10:       $\langle a_i, p_i \rangle \sim P_i$ 
11:      if  $RES^{rem}(a_i) \leq RES^{req}(\phi_i)$  then
12:         $SN_{SFC_{k,t}} \leftarrow \text{false}; \text{break}$ 
13:      end if
14:      Append  $s_i, a_i, p_i$  to  $\mathbf{S}_t, \mathbf{A}_t, \mathbf{P}_t$ 
15:    end for
16:    if  $SN_{SFC_{k,t}}$  is false then
17:      continue
18:    end if
19:    Check link mapping for  $SFC_{k,t}$  and set  $SL_{SFC_{k,t}}$ 
20:    if  $SL_{SFC_{k,t}}$  is true then
21:      Embed all VNFs and virtual links of  $SFC_{k,t}$ 
22:    end if
23:    Compute reward vector  $\mathbf{R}_t$  and adjust  $\mathbf{R}_t \leftarrow \vec{f} \odot \mathbf{R}_t$ 
24:    Store  $\langle \mathbf{S}_t, \mathbf{A}_t, \mathbf{R}_t, \mathbf{P}_t \rangle$  into  $B$ 
25:  end for
26:  if  $|B| \geq \text{batch\_size}$  then
27:    Compute advantages  $\hat{A}_\tau^{GAE}$  and value targets  $\Upsilon_\tau$ 
28:    for epoch = 1 to  $N_{\text{epoch}}$  do
29:      Shuffle  $B$  and split into mini-batches  $\{b\}$ 
30:      for each mini-batch  $b$  do
31:        Calculate the loss according to Eq.(24)
32:        Update parameters according to Eq.(25)
33:      end for
34:    end for
35:     $B \leftarrow \emptyset$ 
36:  end if
37: end for

```

Difference (TD) error for the state value function $V_\omega(s)$ can be expressed as:

$$\delta_t = r_i^t + \gamma V_\omega(s_{i+1}^t) - V_\omega(s_i^t), \quad (18)$$

where γ is the discount factor. And the policy update process relies on the estimation of the advantage function, which measures the relative improvement in expected returns when executing action a_i^t in state s_i^t , compared to the average action selection under the same policy in the identical state. To compute the multi-step TD advantage function, we employ the Generalized Advantage Estimation (GAE) method, as proposed by Schulman et al. [34]. GAE effectively balances the trade-off between bias and variance by applying an exponentially weighted average to multi-step TD residuals. The advantage function can be computed as follows:

$$A_\tau^{GAE} = \sum_{l=0}^{(k+1)T-1-t} (\gamma\lambda)^l \delta_{t+l}. \quad (19)$$

TABLE II: Simulation parameter setup of physical network

Intra-Layer				
Attribute	$\mathcal{S}$	$\mathcal{A}$	$\mathcal{G}$	Description
$ N $	10	30	100	Number of physical node
$RES_{cpu}^{max}(n_i)$	U[100, 500]	U[100, 500]	U[500, 2000]	CPU capacity (units)
$RES_{mem}^{max}(n_i)$	U[1000, 2000]	U[500, 2000]	U[1000, 4000]	Memory capacity (MB)
$RES_{bw}^{max}(l_i)$	U[500, 1000]	U[1000, 2000]	U[2000, 3000]	Bandwidth capacity (Mbps)
$DLY_{proc}(n_i)$	U[5, 10]	U[5, 10]	U[1, 5]	Processing delay of physical node (ms)
$DLY_{prop}(l_i)$	$\beta^\dagger d\langle n_i, n_j \rangle$	$\beta d\langle n_i, n_j \rangle$	$\beta d\langle n_i, n_j \rangle$	Propagation delay of physical link (ms)
Inter-Layer				
Attribute	$\mathcal{S} \leftrightarrow \mathcal{A}$	$\mathcal{S} \leftrightarrow \mathcal{G}$	$\mathcal{A} \leftrightarrow \mathcal{G}$	
$RES_{bw}^{max}(l_i)$	[500, 1000]	[500, 1000]	[1000, 2000]	Limited bandwidth capacity (Mbps)
$DLY_{prop}(l_i)$	$\beta d\langle n_i \in N^{\mathcal{S}}, n_j \in N^{\mathcal{A}} \rangle$	$\beta d\langle n_i \in N^{\mathcal{S}}, n_j \in N^{\mathcal{G}} \rangle$	$\beta d\langle n_i \in N^{\mathcal{A}}, n_j \in N^{\mathcal{G}} \rangle$	Propagation delay of physical link (ms)

[†] In our experiment, β is set to 0.1

The advantage function is normalized as follows:

$$\hat{A}_\tau^{GAE} = \frac{A_\tau^{GAE} - \text{mean}(A_\tau^{GAE})}{std(A_\tau^{GAE}) + \epsilon}. \quad (20)$$

For each sampled point (i, t) , the probability ratio is introduced to evaluate the relative change in policy probabilities between the updated and the previous policy. This ratio is mathematically expressed as:

$$PR_t(\theta) = \frac{\pi_\theta(a_i^t | s_i^t)}{\pi_{old \theta}(a_i^t | s_i^t)}. \quad (21)$$

In this work, we employ the PPO-clip to ensure the policy updates remain within a predefined range, thereby mitigating the risk of excessively large updates that could destabilize the learning process. The clipped objective function is mathematically expressed as follows:

$$L^\pi(\theta) = \mathbb{E}_\tau \left[\min \left(PR_\tau(\theta) \hat{A}_\tau^{GAE}, \text{clip}(PR_\tau(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_\tau^{GAE} \right) \right]. \quad (22)$$

The objective of Critic network is fitting the returns $\Upsilon_\tau = A_\tau^{GAE} + V_{old \omega}(s_\tau)$. Consequently, we compute the loss of the Critic network using mean squared error (MSE), which is formulated as follows:

$$L^V(\omega) = \mathbb{E}_\tau [(V_\omega(s_\tau) - \Upsilon_\tau)^2]. \quad (23)$$

Subsequently, we can derive the composite surrogate objective, with the overall surrogate loss being calculated as follows:

$$L(\theta, \omega) = - \left(L^\pi(\theta) + c_1 \frac{1}{|\tau|} \mathbb{S}[a_\tau] \right) + c_2 L^V(\omega) \quad (24)$$

where, c_1 and c_2 represent the value loss coefficient and the entropy coefficient, respectively. The term $\mathbb{S}[a] = \mathbb{E}_{a \sim \pi_\theta} [-\log \pi_\theta(a|s)]$ is defined as the entropy reward, which encourages the agent to explore thoroughly and prevents premature convergence of the policy network.

Typically, the update magnitude of the value network is larger than that of the policy network. We use different learning rates to update the policy and value networks. The gradient update process is as follows:

$$\theta \leftarrow \theta - \alpha_\pi \nabla_\theta L(\theta, \omega), \quad (25a)$$

$$\omega \leftarrow \omega - \alpha_V \nabla_\omega L(\theta, \omega). \quad (25b)$$

2) *Train Steps*: The overall training pipeline of the PPO-based SFC embedding algorithm is summarized in Algorithm 1 and Figure 2. At the beginning of each episode, the environment is reset and the controller sequentially processes the SFC requests in SFCR. For a given $SFC_{k,t}$, the actor network traverses its VNFs and selects physical nodes according to the current network state, while a feasibility flag $SN_{SFC_{k,t}}$ is used to reject requests whose node capacities are violated. If all VNFs pass the node-level check, a second-stage link mapping is performed and its result is recorded by $SL_{SFC_{k,t}}$. According to the outcomes of node and link mapping, a trajectory $(\mathbf{S}_t, \mathbf{A}_t, \mathbf{R}_t, \mathbf{P}_t)$ is constructed for $SFC_{k,t}$, where the reward vector $\mathbf{R}_t$ is further adjusted by the symbol vector f defined in Eq.(15) before being stored in the experience buffer B . Once the buffer size $|B|$ reaches the predefined batch size, the collected trajectories in B are used to update the actor and critic networks. Generalized Advantage Estimation is applied to obtain the advantages $\hat{A}_\tau^{GAE}$ and the corresponding value targets Υ_τ . Then PPO performs N_{epoch} epochs of mini-batch optimization: in each epoch, B is shuffled and split into mini-batches, and the policy loss and value loss are computed according to Eq.(24), followed by parameter updates as in Eq.(25). After the optimization, the buffer B is cleared and the algorithm proceeds to the next round of interaction.

TABLE III: Configurations of SFCR

Attribute	Value	Description
$ N' $	U[2, 10]	Number of virtual nodes
T_{occu}	U[200, 400]	Fixed time units
$RES_{cpu}^{req}(n'_i)$	U[60, 200]	Required CPU (units)
$RES_{mem}^{req}(n'_i)$	U[100, 400]	Required memory (MB)
$RES_{BW}^{req}(l'_i)$	U[50, 300]	Required bandwidth (Mbps)

V. EXPERIMENT AND PERFORMANCE EVALUATION

A. Setup of Experiment

1) *Simulation network*: To represent the hierarchical network structure of SAGIN structurally, we randomly generate three waxman graph to represent $\mathcal{S}, \mathcal{A}, \mathcal{G}$ layers respectively and the connectivity of nodes within same domain depends on the euclidean distance related probability. For inter-domain connectivity, we randomly select edge nodes from different

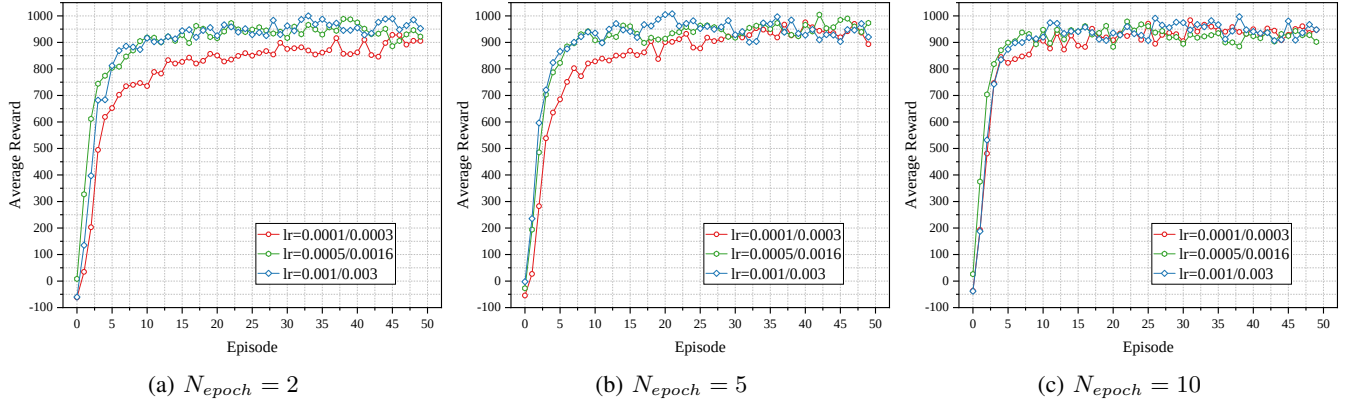


Fig. 3: Reward curves at different learning rates (α_π/α_V) with different $N_{epoch} \in \{2, 5, 10\}$.

domain and calculate the connectivity obey above probability low. The generated three layer waxman graph obey the characteristics of real-world networks, where latency increases and connection probability decreases as distance increases. Table II shows the simulation parameter settings for multi-layer physical networks. There are differences in the number of physical nodes, resource capacity, and latency parameters at each level to reflect the differences in computing and communication capabilities in the hierarchical structure. In terms of intra layer, the number of physical nodes in the three layers is 10, 30, and 100, respectively. The resources (CPU, memory, bandwidth) of each node are randomly allocated according to a uniform distribution within a specified range (denoted as $U[a,b]$). The processing delay of nodes also varies according to different levels, and the propagation delay of links is calculated according to the formula $\beta \cdot d\langle n_i, n_j \rangle$, where $d\langle n_i, n_j \rangle$ represents the Euclidean distance between nodes. In terms of inter layer, including $L^{S \leftrightarrow A}$, $L^{S \leftrightarrow G}$ and $L^{A \leftrightarrow G}$, the bandwidth and propagation delay of the three types of cross layer connections are also limited, which is used to simulate the limited communication resources between layers in real networks. To evaluate the performance of the proposed SFCR algorithm under different resource requirements, we set the basic configuration parameters according to Table III. The number of virtual nodes $|N'|$ varies between $[2, 10]$, used to simulate VNE problems of different scales. The T_{occu} in the range of $[200, 400]$ denotes the cooling period after the substrate resource has been allocated to an SFC, indicating that during this interval, the corresponding resource cannot be reassigned to other SFCs. The required CPU and memory capacity for each virtual node are $RES_{cpu}^{req}(n'_i) \in [60, 200]$ units and $RES_{mem}^{req}(n'_i) \in [100, 400]$ MB, respectively; Link bandwidth requirement $RES_{bw}^{req}(l'_i)$ value $[50, 300]$ Mbps.

B. Convergence Analysis

During the training phase, the request arrival rate η is fixed at 0.6, with the model update batch size set to 4096 and the mini-batch size set to 256. The discount factor and GAE hyperparameter are assigned the conventional values of 0.99 and 0.95, respectively, while the loss coefficients c_1 and c_2 are set to 0.01 and 0.5. Figure 3 provides a comprehensive analysis of

the proposed method's effectiveness. We evaluate the learning trends under different learning rates (α_π/α_V : 0.001/0.003, 0.0005/0.0016, 0.0001/0.0003) and different update frequencies ($N_{epoch} = 2, 5, 10$). The results indicate that lower learning rates or fewer update iterations slow down the convergence process. However, during training, the model consistently converges to the optimal policy in fewer than 20 episodes, regardless of the hyperparameter settings. Moreover, as the training iterates, the model performance remains stable within a very small range, demonstrating strong robustness and consistency.

C. Baseline

To validate the effectiveness of our proposed algorithm, we compare it with the following baseline algorithms:

- Random: This baseline employs a two-stage embedding strategy. Physical nodes are randomly selected under constraints to host virtual nodes, while link mapping is performed using Dijkstra. This method evaluates embedding performance under policy-free.
- NRM [35]: A heuristic two-stage embedding algorithm calculates node scores based on each node's resources and bandwidth. Virtual nodes with the highest rank are greedily mapped to physical nodes with the highest rank, and link mapping is conducted using a resource-constrained shortest path algorithm.
- DRL-SFCP [32]: This method leverages the GCN to simultaneously exploit the non-Euclidean structural characteristics of the physical network and the sequential information of SFC requests. Additionally, the asynchronous advantage actor-critic (A3C) algorithm is employed to enable parallel training of the policy network and value function.
- RLVNE [36]: This work introduces a policy gradient reinforcement learning agent for virtual network embedding, where the policy network observes the substrate network state and outputs node selection decisions trained on historical request traces.
- CEVCN-SFC [37]: This method proposes a deep reinforcement learning-assisted SFC embedding algorithm that employs a five-layer policy network to perceive the network environment and determine node mapping, followed by breadth-first search to complete link embedding.

D. Comparative Analysis

During the testing phase, the system operates continuously for $T_\eta = 10000$ time units at a fixed request arrival rate η , and the $ACCR$, $\overline{LRCR}$, and $\overline{DLY}$ metrics are collected over the entire period T . Upon completion of test T_η , the request arrival rate is increased to $\eta + 0.1$, and the system continues to execute for $T_{\eta+0.1} = 10000$ time units.

1) *Performance of Metric $ACCR$* : As shown in Figure 4, when η is below 0.3, both the proposed method and the NRM algorithm maintain high acceptance rates, with average $ACCR$ of 98.990% and 95.346%, respectively. In contrast, the DRL-SFCP, Random, RLVNE, and CEVCN-SFC methods achieve average acceptance rates of 84.950%, 68.492%, 81.832%, and 83.168%, respectively, for $\eta \leq 0.4$. As the request volume increases, all methods exhibit a declining trend in acceptance rate, which is attributed to the limited SAGIN resources under intensified SFC competition condition, that is only a finite number of SFCs can be embedded. Nevertheless, the proposed method consistently maintains the highest embedding rate under higher arrival rates, demonstrating that it effectively enhances resource-awareness capability. Under high-load scenarios ($\eta \geq 0.5$), the proposed method improves $ACCR$ by an average of 6.991% compared to CEVCN-SFC. Moreover, CEVCN-SFC performs poorly under light-load conditions, indicating that its adaptability across various load scenarios is inferior to that of our method.

2) *Performance of Metric $\overline{DLY}$* : As illustrated in Figure 5, it is first observed that the NRM method achieves relatively poor $\overline{DLY}$ performance in most tests. Combined with the results in Figure 4, this indicates that the heuristic-based NRM has a clear advantage under low-load conditions; however, as resource contention intensifies, such static RA algorithms tend to fall into local optima, leading to degraded performance. Furthermore, for the Random curve, it is observed that as the number of arriving SFCs increases, the $\overline{DLY}$ paradoxically decreases. This phenomenon suggests that SFCs containing more VNFs are more difficult to embed under resource-scarce conditions, and a greater number of VNFs implies more complex SFC topologies and inherently higher delays. The proposed method consistently achieves leading performance in terms of $\overline{DLY}$. Specifically, when $\eta \leq 0.6$ the proposed method significantly outperforms the other baselines. When $0.7 \leq \eta \leq 1.0$, the $\overline{DLY}$ performance of the proposed method is comparable to that of the best baseline (CEVCN-SFC); however, overall, the proposed method reduces $\overline{DLY}$ by 13.556% compared to CEVCN-SFC.

3) *Performance of Metric $\overline{LRCR}$* : Figure 6 intuitively reflects two key insights. First, under the same load environment, the proposed method achieves a significantly higher $\overline{LRCR}$, indicating that our method embeds a greater number of SFC requests. Second, to achieve the same $\overline{LRCR}$ level, our method requires a lower SFC arrival rate. Because η remains constant during testing and is proportional to the SFC arrival volume, this demonstrates that the embedded SFCs yield maximum individual benefits. As shown in Figure 6, compared to CEVCN-SFC, DRL-SFCP, NRM, Random, and RLVNE, the proposed method achieves the highest $\overline{LRCR}$ in all tests, representing an average improvement of 27.763% over CEVCN-SFC. It is

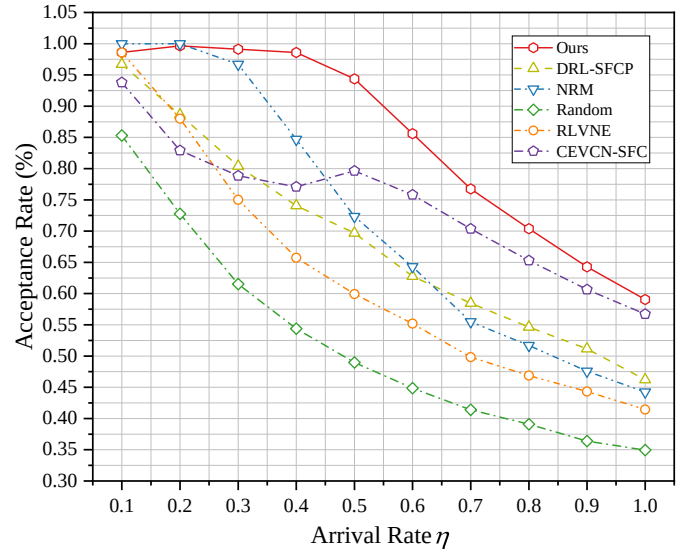


Fig. 4: Performance comparison of $ACCR$ under varying request arrival rate η .

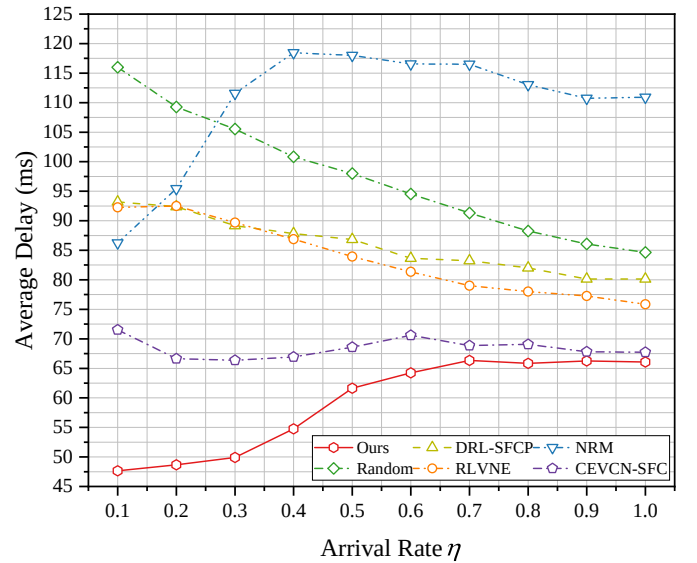


Fig. 5: Performance comparison of $\overline{DLY}$ under varying request arrival rate η .

observed that the CEVCN-SFC, RLVNE, NRM, and Random algorithms exhibit a pronounced upward trend in $\overline{LRCR}$ under high-load conditions. This is because these methods do not fully utilize network resources, resulting in some requests that should have been successfully embedded being rejected due to model errors. Correspondingly, the proposed method reaches a relatively high $\overline{LRCR}$ level around $\eta = 0.6$, with a slower rate of increase thereafter. This precisely demonstrates that our method is more equitable and can comprehensively consider each arriving SFC request while maximizing the utilization of available network resources.

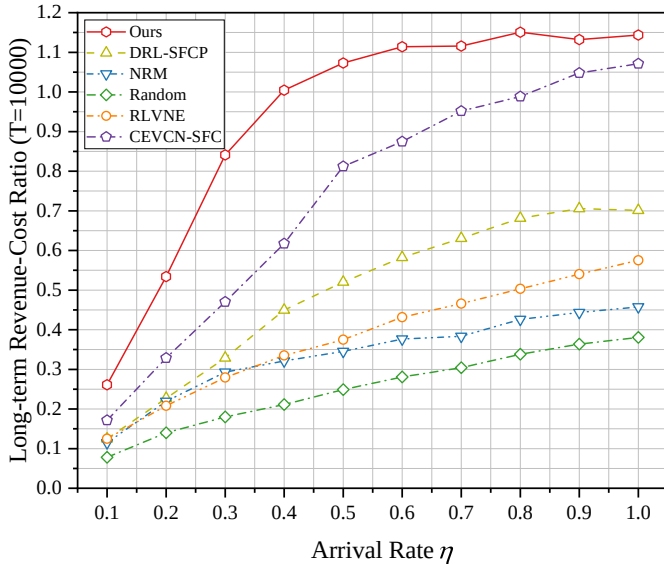


Fig. 6: Performance comparison of $\overline{LRCR}$ under varying request arrival rate η .

VI. CONCLUSION

In this work, we investigated the energy-efficient deployment and joint resource allocation of SFC within SAGIN enhanced by MEC. To address the high-dimensional complexity and dynamic resource constraints of such environments, we proposed a novel GRL framework that integrates GCN with PPO. This design enables efficient extraction of spatial-topological features and adaptive decision-making across heterogeneous network layers. Extensive simulation results show that our framework significantly improves the SFC embedding success rate, reduces E2E service latency, and achieves better load balancing compared to existing baselines. These outcomes highlight the practicality and scalability of the proposed method for future intelligent SAGIN-MEC systems.

REFERENCES

- [1] S. J. Hussain Pirzada, A. Murtaza, T. Xu, and L. Jianwei, "Disaster management using ip-based space-air-ground information network," in *2019 IEEE International Conference on Unmanned Systems (ICUS)*, 2019, pp. 119–123.
- [2] R. P. Marinho, L. F. M. Vieira, M. A. M. Vieira, and A. A. F. Loureiro, "Sagin-cain: A 3d routing protocol for post-disaster sagin 6g network," in *2024 20th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*, 2024, pp. 685–692.
- [3] H. Zhang, M. Jiang, L. Ma, Z. Xiang, and S. Zhuo, "Computing offloading strategy for internet of medical things in space-air-ground integrated network," in *2023 IEEE International Conference on E-health Networking, Application & Services (Healthcom)*, 2023, pp. 177–182.
- [4] P. Zhang, Y. Zhang, N. Kumar, M. Guizani, A. Barnawi, and W. Zhang, "Energy-aware positioning service provisioning for cloud-edge-vehicle collaborative network based on drl and service function chain," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 3424–3435, 2024.
- [5] Q. Chen, Z. Guo, W. Meng, S. Han, C. Li, and T. Q. S. Quek, "A survey on resource management in joint communication and computing-embedded sagin," *IEEE Commun. Surveys Tuts.*, pp. 1–1, 2024.
- [6] J. Du, J. Wang, A. Sun, J. Qu, J. Zhang, C. Wu, and D. Niyato, "Joint optimization in blockchain- and mec-enabled space-air-ground integrated networks," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 31 862–31 877, 2024.
- [7] J. Zhang, Z. Wang, N. Ma, T. Huang, and Y. Liu, "Enabling efficient service function chaining by integrating nfv and sdn: Architecture, challenges and opportunities," *IEEE Netw.*, vol. 32, no. 6, pp. 152–159, 2018.
- [8] Y. Gao, Z. Ye, and H. Yu, "Cost-efficient computation offloading in sagin: A deep reinforcement learning and perception-aided approach," *IEEE J. Sel. Areas Commun.*, vol. 42, no. 12, pp. 3462–3476, 2024.
- [9] M. Sasabe and T. Hara, "Capacitated shortest path tour problem-based integer linear programming for service chaining and function placement in nfv networks," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 1, pp. 104–117, 2021.
- [10] G. Wang, S. Zhou, S. Zhang, Z. Niu, and X. Shen, "Sfc-based service provisioning for reconfigurable space-air-ground integrated networks," *IEEE J. Sel. Areas Commun.*, vol. 38, no. 7, pp. 1478–1489, 2020.
- [11] P. Zhang, K. Yan, N. Kumar, L. Tan, M. Guizani, K. I. Kostromitin, J. Wang, and J. Zhang, "A service function chain mapping scheme based on functional aggregation in space-air-ground integrated networks," *Journal of Network and Computer Applications*, vol. 224, p. 103829, 2024.
- [12] F. Manessi, A. Rozza, and M. Manzo, "Dynamic graph convolutional networks," *Pattern Recognition*, vol. 97, p. 107000, 2020.
- [13] J. Li, W. Shi, H. Wu, S. Zhang, and X. Shen, "Cost-aware dynamic sfc mapping and scheduling in sdn/nfv-enabled space-air-ground-integrated networks for internet of vehicles," *IEEE Internet Things J.*, vol. 9, no. 8, pp. 5824–5838, 2022.
- [14] L. He, Z. Jia, K. Guo, H. Gan, Z. Han, and C. Yuen, "Online joint data offloading and power control for space-air-ground integrated networks," *IEEE Wireless Commun.*, vol. 23, no. 12, pp. 18 126–18 141, 2024.
- [15] L. He, J. Li, Y. Wang, J. Zheng, and L. He, "Balancing total energy consumption and mean makespan in data offloading for space-air-ground integrated networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 1, pp. 209–222, 2024.
- [16] Y. Liu and J. Zhang, "Service function chain embedding meets machine learning: Deep reinforcement learning approach," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 3, pp. 3465–3481, 2024.
- [17] Y. Zhu, H. Yao, T. Mai, W. He, N. Zhang, and M. Guizani, "Multiagent reinforcement-learning-aided service function chain deployment for internet of things," *IEEE Internet Things J.*, vol. 9, no. 17, pp. 15 674–15 684, 2022.
- [18] T. Subramanya, D. Harutyunyan, and R. Riggio, "Machine learning-driven service function chain placement and scaling in mec-enabled 5g networks," *Computer Networks*, vol. 166, p. 106980, 2020.
- [19] L. He, B. Liang, J. Li, and M. Sheng, "Joint observation and transmission scheduling in agile satellite networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 12, pp. 4381–4396, 2022.
- [20] Y. Cao, Z. Jia, C. Dong, Y. Wang, J. You, and Q. Wu, "Sfc deployment in space-air-ground integrated networks based on matching game," in *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2023, pp. 1–6.
- [21] Z. Jia, Y. Cao, L. He, Q. Wu, Q. Zhu, D. Niyato, and Z. Han, "Service function chain dynamic scheduling in space-air-ground integrated networks," *IEEE Trans. Veh. Technol.*, vol. 74, no. 7, pp. 11 235–11 248, 2025.
- [22] P. Zhang, P. Yang, N. Kumar, and M. Guizani, "Space-air-ground integrated network resource allocation based on service function chain," *IEEE Trans. Veh. Technol.*, vol. 71, no. 7, pp. 7730–7738, 2022.
- [23] Y. Zhang, R. Wang, J. Hao, Q. Wu, Y. Teng, P. Wang, and D. Niyato, "Service function chain deployment with vnf-dependent software migration in multi-domain networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 5, pp. 3685–3702, 2025.
- [24] R. Wang, X. Yu, Q. Wu, C. Yi, P. Wang, and D. Niyato, "Efficient deployment of partial parallelized service function chains in cpu+du-based heterogeneous nfv platforms," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9090–9107, 2024.
- [25] Q. Xia, G. Wang, Z. Xu, W. Liang, and Z. Xu, "Efficient algorithms for service chaining in nfv-enabled satellite edge networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 5677–5694, 2024.
- [26] S. Yang, F. Li, Z. Zhou, X. Chen, Y. Wang, and X. Fu, "Online control of service function chainings across geo-distributed datacenters," *IEEE Trans. Mobile Comput.*, vol. 22, no. 6, pp. 3558–3571, 2023.
- [27] G. Zheng, N. Wang, P. Qian, D. Griffin, and R. R. Tafazolli, "Sdn-based service function chaining in integrated terrestrial and leo satellite-based space internet," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 2, pp. 537–550, 2025.
- [28] S. Wang, H. Cao, L. Yang, S. Garg, G. Kaddoum, and M. Alrashoud, "Gcn-based multi-agent deep reinforcement learning for dynamic service

function chain deployment in iot,” *IEEE Trans. Consum. Electron.*, vol. 70, no. 3, pp. 6105–6118, 2024.

- [29] T. Wang, Q. Fan, C. Wang, L. Yang, L. Ding, N. J. Yuan, and H. Xiong, “Flagvne: a flexible and generalizable reinforcement learning framework for network resource allocation,” in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, ser. IJCAI ’24, 2024.
- [30] Q. Fan, P. Pan, X. Li, S. Wang, J. Li, and J. Wen, “Drl-d: Revenue-aware online service function chain deployment via deep reinforcement learning,” *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 4531–4545, 2022.
- [31] J. Zhang, H. Yu, G. Fan, Q. Tang, Z. Li, and J. Xu, “Cold-start-aware cloud-native parallel service function chain caching in edge-cloud network,” *IEEE Internet Things J.*, vol. 11, no. 11, pp. 20 340–20 356, 2024.
- [32] T. Wang, Q. Fan, X. Li, X. Zhang, Q. Xiong, S. Fu, and M. Gao, “Drl-sfcp: Adaptive service function chains placement with deep reinforcement learning,” in *ICC 2021 - IEEE International Conference on Communications*, 2021, pp. 1–6.
- [33] N. M. M. K. Chowdhury, M. R. Rahman, and R. Boutaba, “Virtual network embedding with coordinated node and link mapping,” in *IEEE INFOCOM 2009*, 2009, pp. 783–791.
- [34] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” in *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2016.
- [35] P. Zhang, H. Yao, and Y. Liu, “Virtual network embedding based on computing, network, and storage resource constraints,” *IEEE Internet Things J.*, vol. 5, no. 5, pp. 3298–3304, 2018.
- [36] H. Yao, X. Chen, M. Li, P. Zhang, and L. Wang, “A novel reinforcement learning algorithm for virtual network embedding,” *Neurocomputing*, vol. 284, pp. 1–9, 2018.
- [37] P. Zhang, Y. Zhang, N. Kumar, M. Guizani, A. Barnawi, and W. Zhang, “Energy-aware positioning service provisioning for cloud-edge-vehicle collaborative network based on drl and service function chain,” *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 3424–3435, 2024.



remote sensing, satellite positioning, and collaborative localization methods, particularly in non-cooperative satellite system.

Jian Fan received the B.E. degree from Beijing Institute of Technology, Beijing, China, in 2014, and the M.S. degree from Beihang University, Beijing, China, in 2022. In November 2025, he joined Beijing University of Posts and Telecommunications, Beijing, China, where he is currently working. Meanwhile, he is pursuing the Ph.D. degree at Beihang University, Beijing, China, starting in 2024. From 2014 to 2019 and from 2022 to 2024, he worked as a researcher at the Beijing Institute of Remote Sensing Information Technology, Beijing, China. His research focuses on



management and optimization.

Lizhuang Tan is currently an Associate Professor with Shandong Provincial Key Laboratory of Computer Networks, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences). He received his Ph.D. degree from School of Electronic and Information Engineering, Beijing Jiaotong University in 2022. He has published more than 20 journal or conference papers, such as IEEE TCC, IEEE TNSM, ELSEVIER CN, APNOMS, etc. His research interests include network measurement,



Peiyang Zhang is currently a Professor with the College of Computer Science and Technology, China University of Petroleum (East China). He received his Ph.D. in the School of Information and Communication Engineering at University of Beijing University of Posts and Telecommunications in 2019. He has published multiple high quality papers since 2016, such as IEEE T-IFS, IEEE TSC, IEEE T-ITS, IEEE TMC and etc. He served as the Technical Program Committee of IJCAI’25, AAI’25, AAI’24, AAI’23. He is the editorial board of Scientific

Reports, Drones, CMC-Computers, Materials & Continua, Mobile Information Systems, International Journal of Computational Intelligence Systems and Artificial Intelligence and Applications (AIA). His research interests include future internet architecture, network virtualization, and artificial intelligence for networking.



Chunxiao Jiang (IEEE Fellow) is a national outstanding youth, and an associate professor in the School of Information Science and Technology, Tsinghua University. He received the B.S. degree from Beihang University in 2008 and the Ph.D. degree from Tsinghua University in 2013, both with the highest honors. His research interests include game theory, optimization, and statistical methods for communication and networking, especially in space and heterogeneous networks. Dr. Jiang serves as an editor for several IEEE journals, including IEEE

Internet of Things Journal, IEEE Network, and IEEE Communications Letters, and as a guest editor for others. He has held various roles in international conferences such as symposium chair and publication chair. His work has received multiple best paper awards from IEEE conferences and societies. He was also honored with the IEEE VTS Early Career Award and national awards including the Second Prize of the National Technical Invention Award and the Excellent Young Scientists Fund from NSFC.



Shengpeng Chen is currently working toward the graduate degree with the College of Computer Technology, China University of Petroleum (East China). His research interests include network virtualization and artificial intelligence for networking.